CAD Forge — Backend Interview Prep

Text-to-CAD: natural language → parametric 3D solids. 60-minute cheat sheet aligned to YOUR resume and the actual codebase.

How to use this: The first two pages are a 3-minute pitch. Pages after that are Q&A. If they only ask about the resume bullets, stay on pages 1–4. If they say "agentic / LLM / scale", use pages 5–7.

CRITICAL: Do not claim a LangGraph multi-agent system. README and code: LLM is a parameter extractor; beam/BOM/drawings are deterministic. LangGraph is explicitly deferred.

1. 45-second pitch (memorize this)

I built a FastAPI service that turns an English prompt into a real CAD solid. The language model never draws geometry. It only emits a constrained JSON schema of shape, dimensions, holes, fillets, and text. A validator rejects illegal sizes. A CadQuery kernel then runs a fixed operation order: primitive → holes → shell → fillet/chamfer → text. We export STL, GLB, and STEP in parallel, store a versioned session for undo/redo, and can skip the LLM entirely with a JSON endpoint. Simple prompts never hit the network — regex heuristics plus an in-memory cache.

2. Map each resume bullet to the code (so you don't oversell)

Resume claim	What you actually built	Say this if probed
Text-to-CAD / NL → parametric 3D	POST /generate → extract_params → validate_geometry → build_final_model → STL/GLB/STEP	End-to-end product: parse, constrain, kernel, visualize, export.
Rule-based NLP parsing pipeline	Hybrid. (1) Regex heuristics for box/cylinder/sphere/compounds. (2) Groq llama-3.3-70b JSON extract. (3) Local edits like "10mm taller". Cache SHA1 of prompt.	"Rule-based first; LLM only when heuristics fail. Schema is the contract."
Primitives + holes, fillet, chamfer, text	10 primitives + 16 compound builders. Polar/linear hole patterns. Fillet/chamfer on all edges. Emboss/engrave via CadQuery text, max 24 chars.	CSG: fuse/cut/shell. Fillet BEFORE text for speed.
Modular geometry pipeline / realtime / export	SHAPE_BUILDERS dict + sequential features. Viewer uses GLB. Exports STL (mesh), STEP (BRep), ZIP pack + params JSON.	Params JSON is the source of truth; mesh is a view.

3. System architecture (draw this on a whiteboard)

Client (static HTML/JS) → **FastAPI** → **Parser** (heuristic | Groq JSON | cache | local edit) → **sanitize + validate** → **CadQuery kernel** (Open CASCADE) → **mass props + print estimate** → **export thread pool** (STEP ■ GLB) → **SessionStore** (JSON file, undo cursor).

Side modules (no LLM): Beam Engineer (bending formulas), BOM, 3-view drawings, DFM score, fastener bolt circles, part library, material cost.

Request lifecycle — POST /generate

1. extract_params(prompt) · 2. unknown shape → error + suggestion · 3. missing_fields on primitives → ask user (compounds fill defaults) · 4. validate_geometry (dims, walls, hole vs body, fillet vs min side, 2000 mm cap / 4000 mm furniture) · 5. uuid session_id · 6. generate_model · 7. sessions.save · 8. JSON: params, properties, file URLs.

POST /edit loads previous params, applies local regex or LLM "full JSON rewrite", same validate+rebuild. **POST /generate_manual** is the backend-friendly path: exact params, zero LLM — use this in the interview as your contract/test seam.

4. Geometry kernel — sequential CAD ops (deterministic)

build_final_model is a compiler: JSON in, BRep solid out. Order is intentional for topology and CPU:

Step	Why this order
1. Primitive / compound builder	Untouched solid so face centers/normals stay predictable for later workplanes.

Step	Why this order
2. Circular then square holes (polar or grid)	Cuts on a face workplane. Chair/bed/table skip generic holes (they own their holes).
3. Shell / hollow	Before fillet+text — cheaper, cleaner topology. Optional open face.
4. Fillet or chamfer ALL edges	If done after glyphs, OCCT tries to fillet every letter — slow/fail. So body first.
5. Text last (engrave cut / emboss add)	clean=False ~10× faster; depth sign + mode; fontsize clamped to face width; 24-char cap.

Boolean CSG: holes are extruded tools fused then cut from the body. Text uses combine='cut' or 'a'. Failures in features are often non-fatal (print + skip) so one bad fillet doesn't kill the whole part — mention as a product choice, then admit it can hide bugs.

Parametric vs mesh: STEP is the CAD-native BRep. STL/GLB are tessellations for 3D print and the web viewer. If they ask "why not just generate a mesh with an ML model?" — because dimensions must be exact, editable, and rebuildable. Params JSON + CadQuery is the parametric source of truth.

5. Parsing pipeline — what "NLP" really is

This is **not** a classical NLP stack (no spaCy NER, no CFG compiler). It is a **constrained extraction pipeline** with three fast paths and one slow path:

- **Cache:** SHA1(normalized prompt), 64 entries, FIFO eviction — identical prompts are instant.
- **Heuristic (0 LLM):** regex for box/cylinder/sphere; keyword match for 16 everyday objects; unit conversion mm/cm/m/in.
- **LLM single shot:** Groq JSON mode, temperature 0, system prompt enumerates shapes, required dims, hole patterns, edge_treatment, text_features. One call (old design was normalize+extract = 2 RTTs).
- **Local edit:** "make it 10mm taller", set width/height/diameter, remove hole — no LLM.
- **Offline compounds:** if API key missing, bottle/chair/… still work from defaults.

The LLM output is then _finalize: alias map (desk→table), compound defaults, engrave depth forced negative. That is how you get "deterministic interpretation" despite a stochastic model: **schema + post-process + validator + kernel**.

If they say "your resume says rule-based NLP": "The durable part is rules and schema. The LLM is a fuzzy front-end that I don't trust with numbers for engineering."

6. Backend engineering (this is what a BE interviewer wants)

Topic	Implementation	If they go deeper
API contract	Pydantic request models. Success JSON vs {success:false, errors[]}. Partial params when dims missing.	Idempotency: generate always new session; edit is stateful.
Validation as a gate	Never kernel-run illegal JSON. Walls ≥0.4 mm, fillet < half min side, hole vs body, text depth ≠ 0.	Fail closed on dims; fail open (skip) on some feature exceptions.
Session / history	SessionStore: history[] + cursor. New edit truncates redo branch. File lock + JSON persist.	Single-process lock. Not multi-instance safe. Admit this.
Concurrency	ThreadPoolExecutor(2): STEP export ■ STL→GLB. CadQuery/OCCT is not great under many threads.	Scale-out: queue workers, one kernel per process.
LLM I/O	1 retry, JSON mode fallback, never log API key (/health configured bool only).	Timeouts/circuit breaker not fully productized.
CORS	allow_origins=* — fine for a demo, wrong for production.	Lock origins, auth, signed export URLs.
Testing seam	/generate_manual + params JSON. Heuristics tagged _pipeline.	Golden STL hashes; property-based dim tests.

Why FastAPI + CadQuery (not Three.js-only, not Blender bpy as the kernel)

CadQuery sits on Open CASCADE: real BRep, fillets, STEP. FastAPI is the orchestration layer a backend team understands: typed routes, sync CPU work, static frontend. The UI is not the product; the params schema + kernel is.

7. “Agentic AI” — honest answer (memorize)

If they ask “is this agentic?”, do **not** say yes-and-stop. Use this framing:

> **What we have today:** a *tool-using pipeline* with specialist modules. The LLM has one job: map language → CAD JSON. Downstream “agents” in the README (beam, BOM, drawing, DFM) are **deterministic functions**, not LLM agents. They never invent numbers. That is a feature for manufacturing.
>
> **What “agentic” would mean:** a planner that decomposes “design a printable nameplate that fits this envelope and costs under $2”, calls tools in a loop (extract → DFM → resize → re-export), uses memory, and stops on a success criterion. That is LangGraph/supervisor+tools — explicitly deferred in the repo.
>
> **Why we didn’t agent-loop the kernel:** CAD booleans are expensive and non-associative here (fillet-all-edges is not a feature tree). An agent retrying fillets can hang OCCT. Better: constrained JSON + validator + one-shot kernel.

Degrees of agency (use this ladder)

Level	CAD Forge today	Next if they ask “roadmap”
L0 Chatbot	No — we emit solids + files, not just text.	—
L1 Tool call	Yes: LLM → structured tool args (the JSON schema).	JSON schema / function-calling strictly typed.
L2 Multi-step agent	Partial: /edit is a second step with prior state. No autonomous loop.	LangGraph: extract → DFM critique → auto-fix dims → stop.
L3 Multi-agent	No. Modules are functions, not chatting agents.	Planner + CAD + DFM + cost critic with shared state.

Phrase that sounds senior: “I used the LLM as a probabilistic parser behind a deterministic compiler. Agency without a compiler is a liability in CAD.”

8. Cross-questions they will actually ask

Q: How do you guarantee the model matches the prompt?

You don’t 100%. You guarantee the solid matches the JSON. Confidence comes from: temp=0, JSON mode, required-field list, validator, and heuristic bypass. Ambiguous English (“a small box”) either hits missing_fields or compound defaults. Show the params JSON in the UI so the user can audit.

Q: Why JSON as the intermediate representation?

It is serializable, versionable, editable without the LLM, cacheable, and the session history is just a list of JSON snapshots. That's undo. It's also your API for /generate_manual and tests. A free-form CadQuery script from the LLM would be an RCE and non-reproducible.

Q: Why not let the LLM write CadQuery/Python?

Codegen is unbounded: import os, infinite loops, topology that never returns. A schema is a capability sandbox. Backend answer: least privilege.

Q: How do fillets fail?

Fillet radius larger than local curvature/thickness, or after holes creating tiny edges. We validate radius vs min dimension, then try/except skip. Production should return a structured warning, not a print().

Q: Holes on a cylinder?

Side faces of curved shapes are forced to >Z (flat). Polar pattern on the top of a cylinder is the intended bolt-circle case. Radial holes through the barrel are a known gap.

Q: Units?

Prompt may be cm/m/inch; kernel is always mm. Conversion is in the extractor. Internally never mix units.

Q: How would you test this in CI without Groq?

Golden tests on /generate_manual and heuristics. Mock Groq to return fixtures. Don't snapshot binary STL only — assert volume, bbox, and key params. Optional: hash tessellation with fixed linearDeflection.

Q: How would you scale to 1k concurrent users?

LLM: queue + Groq RPM limits + cache (Redis). Kernel: process pool, not threads (OCCT). Sessions: Postgres, not a JSON file. Object storage for STL/STEP. Don't run CadQuery inside the FastAPI event loop for long jobs — background worker (Celery/RQ/arq). Idempotent job ids.

Q: Is the session store safe?

threading.Lock + rewrite of one JSON file. Fine for a single Uvicorn worker demo. Race across workers, no TTL, no auth, files in /outputs are guessable 8-char ids. Production: DB + auth + signed URLs.

Q: Async FastAPI but CadQuery is blocking?

Correct observation. generate is async def wrapping sync OCCT. Under load it blocks the event loop. Fix: run_in_executor or make those routes sync and use more workers, or a job queue.

Q: What's parametric about this?

The part is defined by named dimensions. Changing height_mm and rebuilding is parametric. It is NOT a full feature tree (no parent-child sketch constraints like SolidWorks). Compounds are scripts with defaults, not a constraint solver.

Q: Difference STL vs STEP?

STL: triangles, 3D print, lossy. STEP: BRep, CAD interchange, exact cylinders stay cylinders. GLB: web viewer.

Q: How does DFM relate?

Separate analyzer on params/solid: thin walls, hole clearance, aspect ratio, text depth. Grade is advice, not a hard gate (except validator mins). Could close the loop agentically later.

Q: Prompt injection?

User text goes into the LLM. Kernel only consumes parsed JSON fields we expect. Still: huge text, unexpected keys. sanitize_params + ignore unknown ops. Never exec() LLM text.

Q: Cost/latency of LLM?

One Groq call, max_tokens ~700, temp 0. Heuristic and cache avoid it. Was 2 calls, now 1. Nameplates: fillet-before-text ~6× faster than fillet-after-glyphs.

Q: Why Groq / llama-3.3-70b?

Low latency JSON, cheap for a demo. Tradeoff vs GPT-4o: more schema misses. Mitigated by JSON mode, aliases, validator. Model is env-configurable.

Q: Compounds vs primitives?

Primitives are generic CSG. Compounds are named recipes (mug = body + handle, etc.) with defaults so "a chair" works. Tradeoff: 16 hand-authored shapes, not open-world generation.

Q: Can it do assemblies / mates?

No. Single solid (or compound as one solid). BOM is a list of session parts, not a constraint assembly.

Q: Material / mass?

Volume from OCCT solid.Volume(); density table; cost = volume × price. Print estimate uses volume + bbox heuristic, not slicer G-code.

9. Current drawbacks (say these before they do — huge credibility)

Gap	Reality in this repo	What you'd do next
Not a full CAD feature tree	Ops are a linear script. Edit "move the hole" is fuzzy. Fillet is all-edges, not selected edges.	Explicit feature list with ids; per-edge selection.
Open-world geometry	10 primitives + 16 compounds. "Turbine blade" or "phone enclosure with snaps" will fail or become a box.	More recipes, or LLM→feature graph with human confirm.
LLM schema drift	Wrong keys, missing pattern type, hallucinated dims. Heuristics don't cover fillets/holes (those force LLM).	Pydantic output model + retry-on-validate + few-shot.
Topology robustness	try/except skip on hole/shell/fillet/text. User may get a part missing the feature they asked for.	Return warnings[] in API; never silent skip.
Sessions / multi-worker	JSON file + process lock. CORS *. 8-char ids in /outputs.	Postgres, auth, object storage, tighter CORS.
Blocking kernel on async routes	OCCT on event loop; no job queue; no progress SSE for long fillets.	Worker queue + job status endpoint.
No true FEA	Beam formulas only (SS/cantilever, point/UDL). Not mesh FEA.	Be honest: closed-form first; FEA later.
Text / fonts	Arial, 24 chars, clean=False. Not true CAM engraving toolpaths.	Outline curves + depth as a proper cut.
No auth / multi-tenant	Local demo. Library and sessions are shared-disk.	User isolation is table-stakes for a product.
Cache invalidation	In-process dict, 64 items, not shared, no TTL on meaning of "box".	Redis keyed by prompt+schema version.
Agent loop missing	Cannot autonomously satisfy DFM then rebuild.	LangGraph: critic tool on DFM grade.
Units / locale	Inch supported in parser; UI is mm-centric.	Display unit preference, store canonical mm.

10. Design tradeoffs (they love these)

- **Schema vs codegen:** less flexible, much safer and testable. Choose schema.
- **One LLM call vs agent loop:** latency and cost; agent loop better for DFM but needs hard stop conditions.
- **Heuristic before LLM:** determinism and $0 for "box 50×30×20"; worse for messy English.
- **Fillet all edges:** one parameter, looks "designed"; cannot fillet only the top rim of a mug without a compound-specific path.
- **Fail-open features:** demo never hard-crashes; manufacturing user may not notice a missing hole — you'd fail-closed in v2.
- **Compounds as Python, not LLM meshes:** a "chair" is always your recipe, so it's printable-ish and parametric; it will never be a designer chair.

11. If they ask you to design v2 on a whiteboard

1) **Canonical Feature Graph** (shape node + feature nodes with ids). 2) **Pydantic v2 schema** as the LLM tool. 3) **Job queue** + object storage. 4) **Redis cache** keyed by schema version. 5) **warnings[]** never silent skip. 6) Optional **LangGraph**: extractor → kernel → DFM tool → if grade < B, propose param patches → max 3 retries → human confirm. 7) Postgres sessions. 8) Don't put LLM on beam math.

12. Resume-risk questions (fix your wording live)

They may read "rule-based NLP" and expect tokenization, POS tags, dependency parse, or a CFG. **Correction in interview:** "I used rule-based extraction and a constrained LLM, not a linguistics NLP pipeline." They may read "agentic" if you say it — only use it as the L1–L2 ladder above. They may read "realtime" — you mean interactive request/response (seconds), not a 60 fps CAD kernel. They may read "diverse geometric configurations" — be concrete: primitives, patterns, compounds; not arbitrary B-rep from a photo.

13. Mini glossary (backend + CAD)

BRep — boundary representation (faces/edges/vertices), what STEP stores. **CSG** — constructive solid geometry (union/cut/intersect). **Workplane** — local 2D sketch plane on a face. **Parametric** — rebuild from dimensions. **Tessellation** — BRep → triangles (STL). **DFM** — design for manufacturability. **FoS** — factor of safety (beam module). **Polar pattern** — N holes on a pitch circle (bolt circle).

14. Closing line

“The interesting backend problem wasn’t generating a mesh. It was putting an untrusted language model behind a schema, a validator, and a deterministic geometry compiler so the output is inspectable, cacheable, and exportable as STEP.”

Files to mention if they want code: app/llm.py (extract/edit/cache), app/validator.py, app/cad_engine.py (build_final_model), app/main.py (/generate /edit /generate_manual), app/session.py.